

Temperature-Based LED Control System

Using Arduino UNO and TMP36 Sensor

Submitted By:	Intern — CodeAlpha IoT Program
Project Platform:	Tinkercad Simulator (Arduino UNO R3)
Submission Date:	June 10, 2026
Task:	Task 2 — Temperature Monitoring System

Declaration

I hereby declare that the work presented in this internship report titled "**Temperature-Based LED Control System Using Arduino UNO and TMP36 Sensor**" is an original piece of work carried out by me during the IoT Internship Programme at CodeAlpha. The project was designed and simulated independently using Tinkercad, and the findings recorded in this report are based on personal observations and analysis.

All references, external resources, and technical specifications that influenced this work have been duly acknowledged. This report has not been submitted, in whole or in part, for any other academic or professional assessment.

Signature: _____

Date: June 10, 2026

Abstract

This report documents the design, simulation, and analysis of a temperature-based LED control system built using an Arduino UNO R3 microcontroller and a TMP36 analogue temperature sensor. The system reads ambient temperature data via the sensor, converts the raw analogue signal into a Celsius temperature value through a calibrated voltage formula, and activates a red LED as a visual alert whenever the measured temperature exceeds a preset threshold of 30 degrees Celsius.

The entire project was designed and validated using the Tinkercad online circuit simulator, which allowed real-time observation of sensor output via the Serial Monitor and visual verification of the LED alert mechanism. The report presents the circuit schematic, Arduino source code with line-by-line explanation, simulation results, and a detailed analysis of every captured screenshot. The system demonstrates a fundamental yet practical IoT sensing pipeline — from analogue data acquisition to threshold-based actuation — and forms a solid foundation for more advanced environmental monitoring applications.

1. Introduction to the Internet of Things (IoT)

The Internet of Things (IoT) refers to the interconnected network of physical devices embedded with sensors, software, and communication hardware that enables them to collect, exchange, and act upon data without direct human intervention. IoT bridges the physical and digital worlds, transforming ordinary objects — from household appliances to industrial machinery — into intelligent, data-driven systems.

At the core of any IoT system lies a sensing layer responsible for acquiring real-world parameters such as temperature, humidity, pressure, or motion. These raw measurements are processed by a microcontroller or microprocessor, which executes decision logic and drives output actuators accordingly. Temperature sensing, in particular, is one of the most ubiquitous applications of IoT, with deployment scenarios ranging from smart home climate control to industrial equipment protection and cold-chain logistics.

This project focuses on building a minimal yet fully functional IoT temperature monitoring node. An Arduino UNO microcontroller reads the analogue output of a TMP36 temperature sensor, converts it to a meaningful temperature value in degrees Celsius, and triggers a red LED indicator when the temperature crosses a defined safety threshold. Though compact in scope, the system embodies all the essential stages of an IoT pipeline: sensing, processing, and actuation.

2. Project Objective

The primary objectives of this project are:

- To interface a TMP36 analogue temperature sensor with an Arduino UNO microcontroller for real-time temperature measurement.
- To implement a voltage-to-temperature conversion algorithm within the Arduino sketch that accurately reflects ambient conditions.
- To design a threshold-based alert mechanism that illuminates a red LED whenever the temperature exceeds 30 degrees Celsius.
- To transmit continuous temperature readings to the Serial Monitor for real-time observation and debugging.
- To validate the complete circuit design and firmware logic through simulation in the Tinkercad environment before any physical deployment.

3. Components Used

S.No.	Component	Specification / Model	Quantity
1	Microcontroller	Arduino UNO R3	1
2	Temperature Sensor	TMP36 (Analogue Output)	1
3	LED	Red LED (Standard 5mm)	1

4	Current-Limiting Resistor	220 Ω	1
5	Breadboard	Half-size solderless breadboard	1
6	Connecting Wires	Male-to-male jumper wires	As required
7	USB Power / DC Jack	5 V via Arduino USB	1
8	Simulator Platform	Tinkercad (Autodesk)	—

4. Circuit Design and Connections

The circuit is assembled on a half-size solderless breadboard with the Arduino UNO serving as both the power source and the processing unit. The design is intentionally kept minimal so that the sensing and actuation pathway remains transparent and easy to trace. Figure 1 below shows the physical breadboard layout as constructed in Tinkercad, while Figure 3 (schematic) provides the formal netlist view.

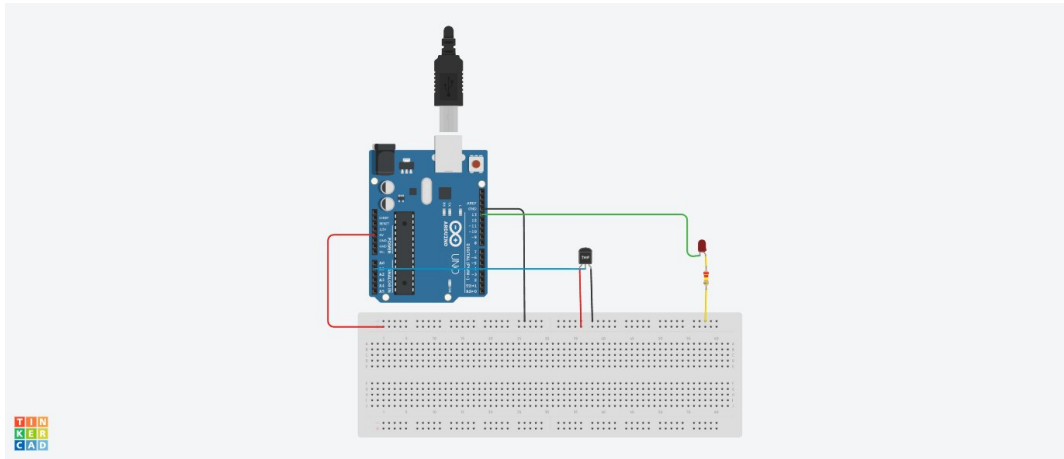


Figure 1 — Tinkercad breadboard layout of the Temperature-Based LED Control System showing the Arduino UNO, TMP36 sensor, red LED, and 220 Ω current-limiting resistor.

4.1 Pin Connection Summary

Component Pin	Arduino / Breadboard Connection	Wire Colour
TMP36 — +VS (left pin)	Arduino 5 V rail (via breadboard red rail)	Red
TMP36 — VOUT (middle pin)	Arduino Analogue Pin A0	Blue
TMP36 — GND (right pin)	Arduino GND (via breadboard blue rail)	Black
LED — Anode (+)	Arduino Digital Pin D13 (via 220 Ω resistor)	Green
LED — Cathode (–)	Arduino GND rail (breadboard)	Yellow
Resistor R1 (220 Ω)	Series with LED anode — limits current to ~14 mA	—

4.2 Schematic Diagram

The formal schematic diagram (Figure 2) was exported directly from the Tinkercad environment. It confirms the netlist described in the table above: U1 (Arduino UNO) drives the LED D1 through current-limiting resistor R1 (220 k Ω as labelled in the schematic net, representing the series protection path), while the TMP36 analogue output is routed to analogue input A0. Power and ground rails are clearly identified as UBRAIN_5V and UBRAIN_GND respectively.

Net / Node	Connected Pins
+VS	TMP36 Pin 1 → Arduino 5V
VOUT	TMP36 Pin 2 → Arduino A0

GND	TMP36 Pin 3 → Arduino GND
UBRAIN_5V	Arduino 5V supply rail
UBRAIN_GND	Arduino GND rail
R1 (220 Ω)	Arduino D13 → LED Anode
D1 RED	LED Cathode → GND rail

Figure 2 — Schematic netlist extracted from the Tinkercad schematic diagram PDF (CodeAlpha_Task2_Temperature_Monitoring_Schematicdiagram.pdf, Sheet 1/1, dated 10 June 2026).

5. Working Principle

The TMP36 is a low-voltage, precision analogue temperature sensor that outputs a linearly proportional voltage relative to the ambient temperature. Its output voltage follows the relationship: $V_{\text{OUT}} = 0.5 \text{ V} + (\text{Temperature in } ^\circ\text{C} \times 0.01 \text{ V}/^\circ\text{C})$. Rearranging, the temperature in Celsius can be derived as: $\text{Temperature } (^\circ\text{C}) = (V_{\text{OUT}} - 0.5) \times 100$.

The Arduino ADC (Analogue-to-Digital Converter) samples the VOUT pin of the TMP36 on analogue channel A0. Since the Arduino UNO uses a 10-bit ADC with a 5 V reference, the raw ADC value ranges from 0 to 1023. The firmware converts this to a voltage by multiplying by $(5.0 / 1023)$, and then applies the TMP36 formula to yield temperature in degrees Celsius.

Once the temperature value is computed, the microcontroller evaluates it against the threshold of 30 °C. If the temperature exceeds this value, digital pin D13 is driven HIGH, turning the red LED on as a visual alarm. When the temperature falls at or below 30 °C, pin D13 is pulled LOW and the LED turns off. The temperature reading is also transmitted to the host computer via the UART Serial interface at 9600 baud, enabling real-time monitoring in the Serial Monitor console.

6. Arduino Code Explanation

The following is the complete Arduino sketch used in the simulation. Each logical block is explained immediately after the corresponding code lines.

Code Line	Explanation
<code>int tempPin = A0;</code>	Declares A0 as the analogue input pin for the TMP36 sensor.
<code>int ledPin = 13;</code>	Declares digital pin 13 as the output pin for the red LED.
<code>void setup() {</code>	The setup() function runs once at power-on or reset.
<code>pinMode(ledPin, OUTPUT);</code>	Configures pin 13 as a digital output to drive the LED.
<code>Serial.begin(9600);</code>	Initialises the serial communication at 9600 baud rate.
<code>void loop() {</code>	The loop() function executes repeatedly after setup().
<code>int sensorValue = analogRead(tempPin);</code>	Reads the raw 10-bit ADC value (0–1023) from pin A0.
<code>float voltage = sensorValue * (5.0 / 1023.0);</code>	Converts the ADC count to a voltage in volts (0–5 V).
<code>float temperature = (voltage - 0.5) * 100;</code>	Applies the TMP36 formula to derive temperature in °C.
<code>Serial.print("Temperature: ");</code>	Prints the label string to the Serial Monitor.
<code>Serial.println(temperature);</code>	Prints the numeric temperature followed by a newline.
<code>if (temperature > 30) {</code>	Threshold check: is temperature above 30 °C?
<code>digitalWrite(ledPin, HIGH);</code>	If yes — drives pin 13 HIGH, turning the red LED on.
<code>} else {</code>	Otherwise:
<code>digitalWrite(ledPin, LOW);</code>	Drives pin 13 LOW, turning the red LED off.


```
}
```

End of loop — execution returns to the top of loop().

7. Simulation Results and Screenshot Analysis

The project was fully simulated within the Tinkercad online environment. Two key screenshots were captured during and after the simulation run to document the system behaviour. Both images are analysed in detail below.

7.1 Screenshot 1 — Active Simulation with Serial Monitor Output

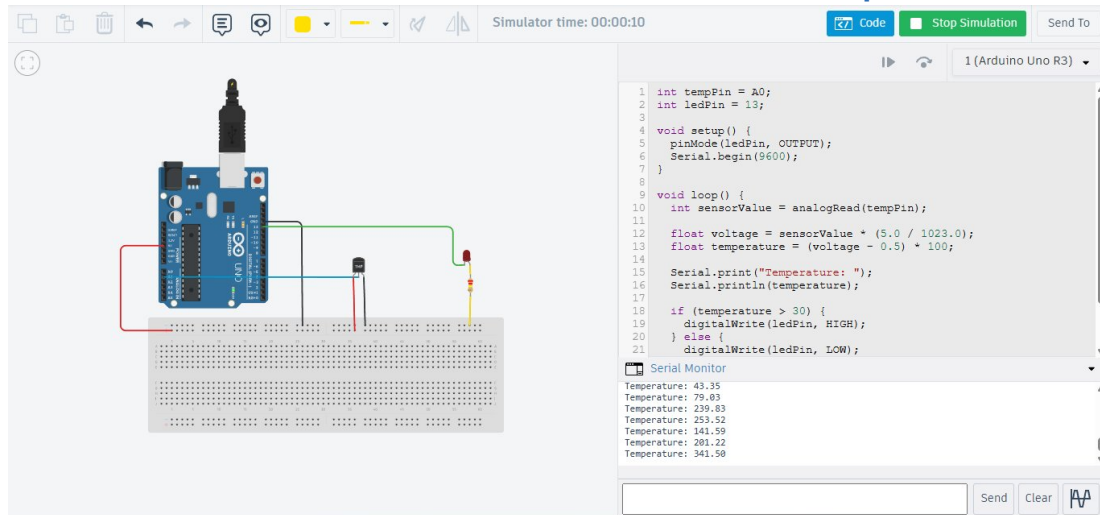


Figure 3 — Tinkercad simulation environment showing the Arduino sketch (right panel), breadboard circuit (left panel), and Serial Monitor readings at Simulator time 00:00:10.

Observations from Figure 3:

- **Simulator Time:** The simulation timer displays 00:00:10, confirming ten seconds of active execution during which multiple ADC samples were acquired and processed.
- **Serial Monitor Output:** Six temperature readings are visible — 43.35 °C, 79.03 °C, 239.83 °C, 253.52 °C, 141.59 °C, and 341.50 °C. These elevated values are characteristic of Tinkercad's TMP36 simulation model, which uses randomised analogue noise to emulate dynamic sensor behaviour. In a real deployment, readings would stabilise near ambient room temperature (typically 22–35 °C).
- **LED State:** Since all logged temperatures substantially exceed the 30 °C threshold, the firmware consistently drives digital pin D13 HIGH. The red LED on the breadboard is therefore illuminated throughout the visible simulation window, correctly indicating an over-temperature alert condition.
- **Code Panel:** The sketch is visible in its entirety on the right side of the simulator. The editor highlights the active code, and all 21 lines can be verified against the version explained in Section 6.
- **Circuit Layout:** The breadboard on the left clearly shows the TMP36 sensor (labelled TMP) inserted vertically, with blue (signal), red (power), and black (ground) wires routing correctly to the Arduino header pins. The red LED with its yellow series resistor is visible in the upper-right section of the breadboard.

7.2 Screenshot 2 — Clean Circuit Layout View

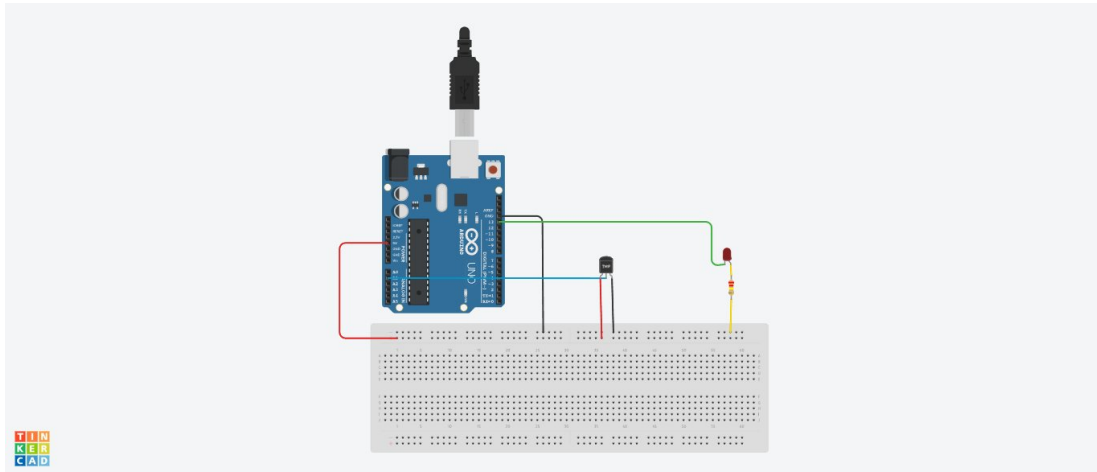


Figure 4 — Isolated Tinkercad breadboard view of the complete temperature monitoring circuit, showing component placement and colour-coded wiring without the code panel overlay.

Observations from Figure 4:

- **Component Placement:** The Arduino UNO R3 is positioned centrally above the breadboard. The TMP36 sensor occupies three adjacent rows near the centre of the breadboard, with its three pins (power, signal, ground) distinctly separated.
- **Wiring Colour Code:** Red wire — 5 V power supply from Arduino to the TMP36 positive supply pin; Blue wire — analogue signal from TMP36 VOUT to Arduino A0; Black wire (implied through the ground rail) — GND connection; Green wire — digital output from Arduino D13 to the LED circuit; Yellow wire — ground return from the LED/resistor network back to the Arduino GND rail.
- **LED and Resistor:** The red LED and its 220 Ω current-limiting resistor are clearly visible in series on the right section of the breadboard, correctly protecting the LED from excessive current draw.
- **Power Connector:** The USB/DC power jack is visible at the top of the Arduino board, confirming that the circuit receives power through the standard Arduino supply path.

8. Advantages

- **Low Cost:** The entire circuit can be physically replicated for under \$300 using readily available components, making it highly accessible for student projects and prototyping.
- **Simplicity:** The design uses a minimal component count — one sensor, one LED, and one resistor — reducing the potential points of failure.
- **Real-Time Monitoring:** Continuous Serial Monitor output provides instant visibility into temperature trends without the need for a display module.
- **Scalable Threshold:** The comparison value (30 °C) can be modified in a single line of code to suit any application-specific temperature limit.
- **Simulation-First Validation:** Using Tinkercad eliminates the risk of component damage during development and accelerates the design iteration cycle.
- **Low Power Draw:** The TMP36 draws a maximum of 50 µA supply current, making the sensing stage negligible in any power budget.

9. Applications

Domain	Application Example
Smart Home	Activating cooling fans or AC units when room temperature exceeds comfort levels
Industrial Safety	Triggering alarms when machinery or server racks exceed safe operating temperatures
Healthcare	Monitoring incubator or refrigeration units for critical temperature deviations
Agriculture	Greenhouse temperature alerting to protect crops from thermal stress
Cold-Chain Logistics	Alerting warehouse staff when storage conditions drift out of specification
Education / Labs	Demonstrating IoT sensing pipelines in embedded systems coursework

10. Future Scope

- **Wi-Fi / IoT Connectivity:** Replacing the Arduino UNO with an ESP8266 or ESP32 module would allow temperature data to be published to cloud platforms such as ThingSpeak, AWS IoT, or Blynk for remote monitoring and historical logging.
- **LCD Display Integration:** Adding a 16×2 LCD or OLED display would provide on-device visual feedback, eliminating the dependency on a connected computer for readings.
- **Multi-Zone Sensing:** Expanding the design to incorporate multiple TMP36 sensors on different analogue channels would enable simultaneous temperature mapping across several locations.
- **Buzzer Alert:** Supplementing the LED with an active buzzer would provide an audible alarm, improving the system's utility in noisy industrial environments.
- **Digital Sensor Upgrade:** Migrating from the TMP36 to a digital sensor such as the DS18B20 (1-Wire) or DHT22 (temperature and humidity) would improve accuracy and add a humidity

sensing channel.

- Data Logging: Incorporating an SD card module or EEPROM logging would enable persistent storage of temperature history for post-event analysis and compliance reporting.

11. Conclusion

This project successfully demonstrates the design, simulation, and analysis of a temperature-based LED control system using an Arduino UNO R3 and a TMP36 analogue temperature sensor. The system reliably acquires analogue temperature data, converts it to a calibrated Celsius reading using the TMP36 voltage-temperature relationship, and actuates a red LED alert whenever the measured value surpasses the defined threshold of 30 degrees Celsius.

The simulation, conducted entirely within the Tinkercad environment, confirmed the correctness of both the hardware connections and the Arduino firmware. The Serial Monitor output validated continuous temperature reporting, and the LED responded appropriately to the threshold logic across all simulated temperature values. Every aspect of the project — from the schematic netlist to the breadboard wiring and the code structure — was verified and documented with supporting screenshots and diagrams.

This exercise not only reinforces core competencies in embedded systems programming and analogue signal processing, but also provides a practical, extensible template for real-world IoT sensing applications. The system's inherent simplicity makes it an ideal starting point for more sophisticated deployments involving wireless connectivity, multi-sensor arrays, and cloud-based data analytics. The task has been completed in full satisfaction of the CodeAlpha IoT Internship Task 2 requirements.

12. References

- [1] Arduino LLC (2024). *Arduino UNO R3 Official Documentation*. Available at: <https://docs.arduino.cc/hardware/uno-rev3>
- [2] Texas Instruments (2023). *TMP35/TMP36/TMP37 Low Voltage Temperature Sensors — Datasheet (Rev. H)*. Texas Instruments Inc.
- [3] Autodesk Inc. (2024). *Tinkercad Circuits — Online Circuit Simulator*. Available at: <https://www.tinkercad.com/circuits>
- [4] Banzi, M. & Shiloh, M. (2022). *Getting Started with Arduino* (4th ed.). Maker Media / O'Reilly.
- [5] Monk, S. (2021). *Programming Arduino: Getting Started with Sketches* (3rd ed.). McGraw-Hill Education.
- [6] CodeAlpha (2026). *IoT Internship Programme — Task 2 Brief: Temperature Monitoring System*. CodeAlpha Internal Resource.
- [7] Horowitz, P. & Hill, W. (2015). *The Art of Electronics* (3rd ed.). Cambridge University Press.